

RepoMind: A Cloud-Native, Event-Driven AWS Pipeline for Code-Aware Repository Q&A

Student Details

- Student Name(s):
 - Moksh Balaji, 240905710
 - Priyanshu Shrivastava, 240905650
- Semester: 5th Semester
- Section: CSE B
- Batch: B2
- Academic Year: 2026-27
- Department: School of Computer Engineering

Supervisor Details

- Supervisor Name: Anil Rana
- Department: School of Computer Engineering

Synopsis / Abstract

RepoMind is a cloud-native system, built entirely on AWS serverless application infrastructure, that lets developers ask natural-language questions about a GitHub repository and receive answers grounded in the actual source code, complete with file and function references. Rather than relying on a single always-on server, the system is designed as an event-driven pipeline: a commit to the connected repository triggers an automated ingestion, code-aware chunking, and embedding workflow orchestrated by AWS Step Functions, while queries are answered on demand through a serverless retrieval-augmented generation (RAG) request path. The entire stack is provisioned through Terraform and deployed through a GitHub Actions CI/CD pipeline, with Amazon CloudWatch monitoring, IAM least-privilege roles, and AWS Secrets Manager for credential handling. All AWS compute, storage, orchestration, and monitoring components are serverless managed services; embedding generation and answer generation are the only external dependencies, consumed as free-tier third-party APIs. The project's primary focus is the cloud architecture, orchestration, and DevOps automation behind the assistant, with the AI/RAG component serving as the applied use case.

Problem Statement

Developers frequently join or navigate unfamiliar codebases and spend significant time manually searching for how specific functionality is implemented, especially without direct access to the original authors. Generic AI chat tools cannot answer these questions accurately because they were never trained on a specific repository and have no way of tracking its ongoing changes. Existing “chat with your codebase” tools are typically closed-source, subscription-based products, leaving no lightweight, transparent, self-deployable alternative for teams or students who want to understand or extend this capability themselves.

This problem affects developers at every level — from students exploring an open-source project to engineers onboarding onto a new team's codebase — and addressing it improves comprehension speed, reduces repeated interruptions to teammates, and lowers the barrier to working productively in unfamiliar code. Within the scope of this mini project, the system targets repositories of the size typical of coursework, open-source utilities, and small team projects, which is the range the serverless ingestion path is designed around.

Proposed Solution

The proposed solution is a cloud-native “Codebase Q&A Assistant” deployed on AWS serverless infrastructure. It consists of six main modules:

Repository Ingestion & Sync Module:

Repository-change detection uses a GitHub Actions workflow triggered on a push to the connected repository. The workflow assumes a dedicated AWS IAM role through GitHub's OIDC identity provider, so no long-lived AWS access keys are stored in GitHub, and starts an AWS Step Functions execution, passing the commit SHA and the list of changed files. An ingestion AWS Lambda function then retrieves the contents of those files through the GitHub REST API rather than cloning the full repository, keeping the ingestion step within Lambda's execution time and temporary storage limits. The end-to-end flow is: GitHub push → GitHub Actions (OIDC) → Step Functions execution → ingestion Lambda.

Code-Aware Chunking & Embedding Module:

Orchestrated by AWS Step Functions, this module classifies each file as code or documentation and splits code along function and class boundaries using

AST-based parsing rather than fixed-size text splitting, so that every chunk is a logically complete unit. Each chunk is stored with the metadata required for traceable answers: repository-relative file path, the enclosing symbol (function or class name) where applicable, and start and end line numbers. Embeddings are generated by calling a free-tier hosted embedding API from an embedding Lambda function invoked through a Step Functions Map state, which batches chunks and bounds concurrency to stay within the provider's rate limits. No model training, GPU compute, or ML hosting infrastructure is required. The data flow is: source files → AST-based code chunks with metadata → embedding API → vector index → Amazon S3.

Serverless Vector Storage & Retrieval Module:

Per-file chunk embeddings and the consolidated serialized vector index are stored as objects in Amazon S3. At query time, a Lambda function loads the serialized index into memory and performs an in-memory similarity search, so no vector database cluster or always-on search service is required. Because embeddings are persisted per file, the index is updated incrementally: on each commit, only files reported as added or modified are re-chunked and re-embedded, entries for deleted files are dropped, and the consolidated index is rebuilt from the stored per-file embeddings. This keeps ingestion cost proportional to the size of the change rather than the size of the repository, while preserving the simplicity of a single serialized index in Amazon S3.

Ingestion State & Metadata Module:

Amazon DynamoDB tracks the state of the system across ingestion runs. It stores per-repository ingestion job records (execution ID, commit SHA, status, timestamps) and per-file index metadata (file path, last-indexed commit, chunk count, embedding object key). The chunking and embedding steps read this table to determine which files actually require re-processing for a given commit and write back the updated state on completion, making DynamoDB the component that enables incremental indexing and job traceability.

AI Query & Answer Generation Module:

The query interface is a Lambda Function URL, which is the HTTPS endpoint configuration of the query Lambda function rather than a separate service. The request path is: user query → Lambda Function URL → query Lambda → query embedding and top-k chunk retrieval from the Amazon S3 index → free-tier LLM API → source-grounded natural-language answer. The response

is returned together with the file paths, symbol names, and line ranges of the chunks the answer was drawn from.

Infrastructure as Code, CI/CD, Monitoring & Security Module:

The complete AWS stack — Lambda functions, the Step Functions state machine, Amazon S3 buckets, the DynamoDB table, IAM roles, and CloudWatch resources — is defined and provisioned through Terraform, with no manually created console resources. A GitHub Actions pipeline validates and plans the Terraform configuration, runs unit tests, packages the Lambda deployment artifacts, and applies the infrastructure, providing a fully automated path from commit to deployed environment. Amazon CloudWatch collects Lambda and Step Functions logs and metrics for pipeline health monitoring and failure diagnosis. Security is handled through IAM least-privilege execution roles scoped per function, GitHub OIDC federation in place of stored AWS credentials, and AWS Secrets Manager for the external embedding and LLM API keys.

Together, these modules address slow codebase comprehension, repeated dependence on teammates for context, and the absence of an accessible, self-deployable alternative to commercial “chat with your code” tools, by combining automated event-driven ingestion, code-aware AI retrieval, and cloud-native, fully automated cloud infrastructure into a single pipeline.

Tools and Technologies

- Cloud Service Provider: Amazon Web Services (AWS)
- Cloud Managed Services: AWS Lambda (ingestion, chunking, embedding, retrieval and answer generation; exposed to users through a Lambda Function URL), AWS Step Functions (ingestion workflow orchestration), Amazon S3 (vector index and embedding storage), Amazon DynamoDB (ingestion job state and index metadata), Amazon CloudWatch (logs, metrics and monitoring), AWS Secrets Manager (external API credentials), AWS IAM (least-privilege roles and GitHub OIDC federation)
- Databases: Amazon DynamoDB (NoSQL — ingestion job state and per-file index metadata)
- Source Version Control: GitHub
- Tools: Terraform (Infrastructure as Code), GitHub Actions (CI/CD), Python (Lambda runtime), AST / tree-sitter (code-aware chunking), a free-tier hosted embedding API (chunk and query embeddings), a free-tier LLM API such as Groq or Gemini (answer generation)

- **Budget & Free-Tier Usage:** All AWS services selected are serverless or usage-metered, so cost scales with invocations rather than uptime and no always-on compute or database instances are provisioned. The project is designed to operate within the AWS Free Tier, and an AWS Budgets budget with email alert thresholds will be configured at the start of the project, before any infrastructure is deployed.

Expected Outcomes

The expected outcome is a working, serverless AWS deployment that allows a developer to query a real GitHub repository in natural language through an HTTPS endpoint and receive accurate, source-referenced answers. The project will be considered successful if a commit to the connected repository automatically triggers the Step Functions ingestion workflow, the code-aware chunking preserves function and class boundaries while retaining file path, symbol, and line-range metadata, the index is updated incrementally so that only changed files are re-embedded, and the assistant returns relevant, traceable answers to a set of representative test questions. Success will also be measured by the completeness of the DevOps implementation — the entire stack reproducibly provisioned from Terraform, a working GitHub Actions CI/CD pipeline, CloudWatch monitoring of the ingestion and query paths, and least-privilege IAM roles — since the primary evaluation focus is the cloud architecture and deployment practices rather than the AI model itself.

Tentative Timeline

Week #	Phase/Tasks
1	Requirement Analysis & Specification; AWS account setup; AWS Budgets alert configuration; target repository selection
2	High Level Design (cloud architecture, AWS managed service selection, embedding and LLM API selection)
3	Low Level Design (Step Functions state machine, DynamoDB schema, chunk metadata schema, Terraform module structure, query API contract)
4	Terraform baseline infrastructure provisioned; GitHub Actions CI/CD pipeline set up and deploying skeleton Lambda functions
5-6	Implementation & Unit Testing: ingestion trigger (GitHub Actions + OIDC), Step Functions workflow, code-aware chunking, DynamoDB state tracking
7	Implementation & Unit Testing: embedding generation, S3 vector

	<i>index construction, incremental update logic</i>
<i>8</i>	<i>Implementation & Unit Testing: retrieval and answer generation via Lambda Function URL; Secrets Manager integration</i>
<i>9</i>	<i>Testing (Pipeline & Integration Testing, debugging); IAM least-privilege hardening; CloudWatch monitoring and alarms</i>
<i>10</i>	<i>Deployment & Release (final deployment through the CI/CD pipeline, end-to-end validation, buffer for fixes)</i>
<i>11</i>	<i>Documentation preparation - Technical Design Document (Introduction, Problem Statement, Solution Architecture, Application Architecture, Technical Designs, Lessons Learned, Individual Contributions, Source Repository Link) and Presentation PPT</i>
<i>12</i>	<i>Final Presentation & Live Demonstration</i>